



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3
Проектування та дослідження програмних систем
зі штучним інтелектом

Тема: «Нейронні мережі прямого розповсюдження для розпізнавання зображення»

Виконала
студентка групи ІК-51мп:
Яковенко Д. О.

ЗМІСТ

Практичне завдання.....	2
Хід роботи.....	2
Обраний датасет і підготовка даних.....	2
Архітектура мережі.....	5
Отримані результати	6
Розпізнавання власних цифр	8
Відповіді на питання	11
Висновки.....	13

Практичне завдання

Завдання: Написати програму що реалізує нейронну мережу прямого розповсюдження для розпізнавання рукописних цифр.

1. Завантажити з tensorflow набір даних MNIST.
2. Обрати архітектуру та запрограмувати нейронну мережу прямого поширення.
3. Провести навчання нейронної мережі на наборі даних MNIST.
4. Провести тестування нейронної мережі тестовій частині набору даних MNIST.
5. Оцінити правильність роботи мережі, побудувати матрицю помилок, обрахувати значення accuracy, precision, recall та F-Score.
6. Провести тестування моделі – написати функцію розпізнавання власноруч написаних цифр.

Хід роботи

Обраний датасет і підготовка даних

Для виконання лабораторної роботи було використано класичний набір даних **MNIST**, який містить 60 000 тренувальних та 10 000 тестових чорно-білих зображень рукописних цифр розміром 28x28 пікселів. Дані були завантажені у вигляді бінарних файлів формату .ubyte та зчитані за допомогою бібліотеки numpy.

Тренувальні дані: (60000, 784) картинок, (60000,) міток
Тестові дані: (10000, 784) картинок, (10000,) міток

Оскільки нейронна мережа прямого поширення (Feedforward Neural Network) приймає на вхід одновимірний масив, кожне зображення було перетворено (flattening) з матриці 28x28 у вектор довжиною 784 елементи.

```
import numpy as np

# Шлях до папки з датасетом
path = '/kaggle/input/mnist-dataset/'

# 1. Функції для читання даних з файлів
def read_images(file_path):
    with open(file_path, 'rb') as f:
        return np.frombuffer(f.read(), np.uint8, offset=16).reshape(-1, 784)

def read_labels(file_path):
    with open(file_path, 'rb') as f:
        return np.frombuffer(f.read(), np.uint8, offset=8)

# 2. Поділ на тренувальні та тестові дані
x_train = read_images(path + 'train-images.idx3-ubyte')
y_train = read_labels(path + 'train-labels.idx1-ubyte')

x_test = read_images(path + 't10k-images.idx3-ubyte')
y_test = read_labels(path + 't10k-labels.idx1-ubyte')

print(f"Тренувальні дані: {x_train.shape} картинок, {y_train.shape} міток")
print(f"Тестові дані: {x_test.shape} картинок, {y_test.shape} міток")
```

Далі було проведено два етапи підготовки даних:

- **Нормалізація:** Значення пікселів (від 0 до 255) були поділені на 255. Це стиснуло діапазон даних до значень від 0 до 1, що необхідно для стабільного та швидкого навчання нейромережі. Тип даних змінено на float32 для оптимізації пам'яті.
- **Категоризація міток:** Цільові мітки (цифри від 0 до 9) були перетворені у бінарні вектори довжиною 10. Це дозволило мережі сприймати цифри як рівноправні класи, а не як кількісні значення.

```
from tensorflow.keras.utils import to_categorical
```

```
# 1. Нормалізація
```

```
x_train = x_train / 255
```

```
x_test = x_test / 255
```

```
print(y_train[:5])
```

```
# 2. Створення категорій
```

```
y_train = to_categorical(y_train, 10)
```

```
y_test = to_categorical(y_test, 10)
```

```
print(f"Нова форма міток: {y_train.shape}")
```

```
print(y_train[:5])
```

```
[5 0 4 1 9]
```

```
Нова форма міток: (60000, 10)
```

```
[[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

```
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
```

```
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]]
```

Архітектура мережі

Було спроектовано послідовну нейронну мережу з такою архітектурою:

- Вхідний шар: Приймає вектор із 784 ознак (пікселів).
- Перший прихований шар: Складається зі 128 нейронів. Використовує функцію активації ReLU, яка дозволяє мережі виявляти нелінійні залежності та уникати проблеми затухання градієнта.
- Другий прихований шар: Складається з 64 нейронів, також з активацією ReLU для формування більш складних ознак на основі попереднього шару.
- Вихідний шар: Містить рівно 10 нейронів (відповідно до кількості класів). Використовує функцію активації Softmax, яка перетворює вихідні значення у вектор ймовірностей (сума яких дорівнює 1).

Для компіляції моделі було обрано оптимізатор Adam (адаптивний алгоритм, що забезпечує швидку збіжність) та функцію втрат Categorical Crossentropy.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense

model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                   epochs=10,
                   batch_size=32,
                   validation_split=0.1)

test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f"\nТочність моделі: {test_accuracy * 100:.2f}%").
```

Навчання моделі проводилося протягом 10 епох. Для контролю перенавчання 10% тренувальних даних було виділено під валідацію.

Отримані результати

Після завершення навчання мережа була протестована на 10 000 відкладених зображень. Для детальної оцінки роботи алгоритму було побудовано матрицю помилок та розраховано ключові метрики, такі як точність, повнота, f1-score.

Epoch 9/10
1688/1688 7s 4ms/step - accuracy: 0.9944 - loss: 0.0174 - val_accuracy: 0.9780 - val_loss: 0.0923
Epoch 10/10
1688/1688 8s 5ms/step - accuracy: 0.9955 - loss: 0.0148 - val_accuracy: 0.9808 - val_loss: 0.0856
313/313 1s 3ms/step - accuracy: 0.9737 - loss: 0.1089

Точність моделі: 97.68%

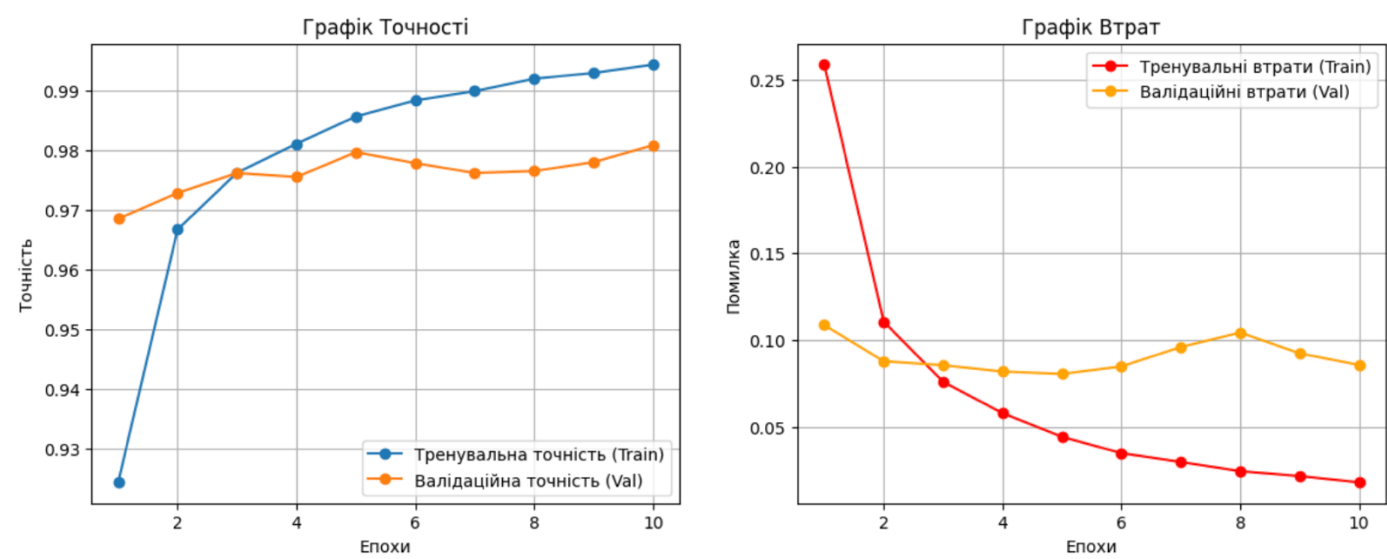


Рис. 1. – Графіки точності і втрат для тренувальних і валідаційних даних (Точності від епохи – ліворуч, Втрат від епохи – праворуч)

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	0.98	0.99	0.99	1135
2	0.97	0.97	0.97	1032
3	0.98	0.98	0.98	1010
4	0.98	0.98	0.98	982
5	0.96	0.98	0.97	892
6	0.99	0.96	0.98	958
7	0.98	0.97	0.98	1028
8	0.96	0.97	0.97	974
9	0.97	0.98	0.97	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

Рис. 2. – Обраховані значення

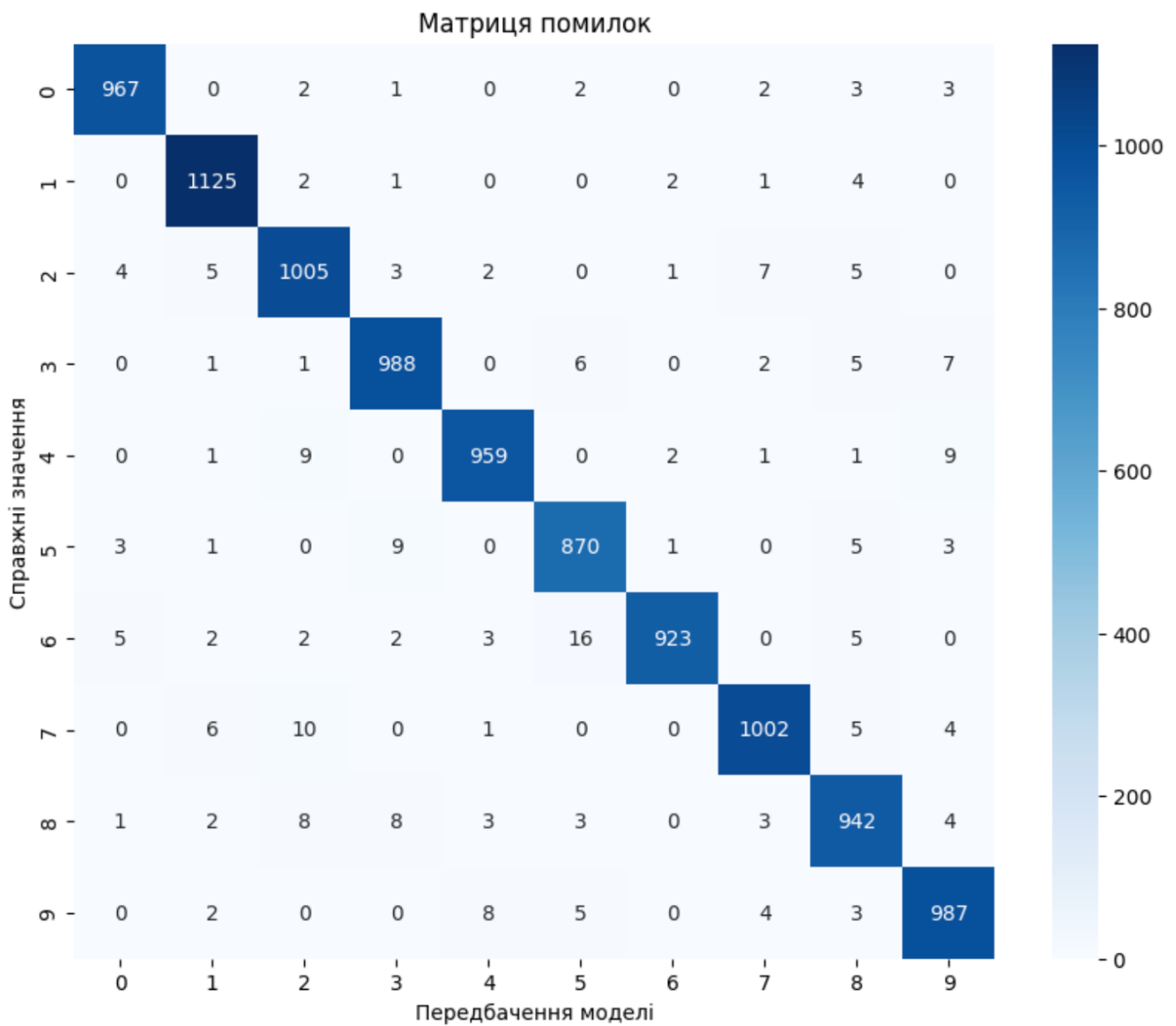


Рис. 3. – Матриця помилок

Для більш глибокого розуміння результатів класифікації було побудовано матрицю помилок. Аналізуючи отриману матрицю, бачимо, що чітко видно діагональну лінію. Це підтверджує, що переважна більшість із 10 000 тестових зображень класифікована абсолютно правильно. Цифри 0 та 1 мають найменшу кількість відхилень. Найчастішою помилкою є те, що справжню цифру 6 мережа 16 разів класифікувала як 5. Це очікувано, оскільки при швидкому написанні нижнє заокруглення цих цифр є майже ідентичним. Справжню цифру 7 модель 10 разів сплутала з 2 (ймовірно, через наявність діагональної лінії).

Розпізнавання власних цифр

Для перевірки здатності моделі до генералізації було розроблено алгоритм розпізнавання власноруч написаних цифр, сфотографованих на камеру та намальованих в Paint.

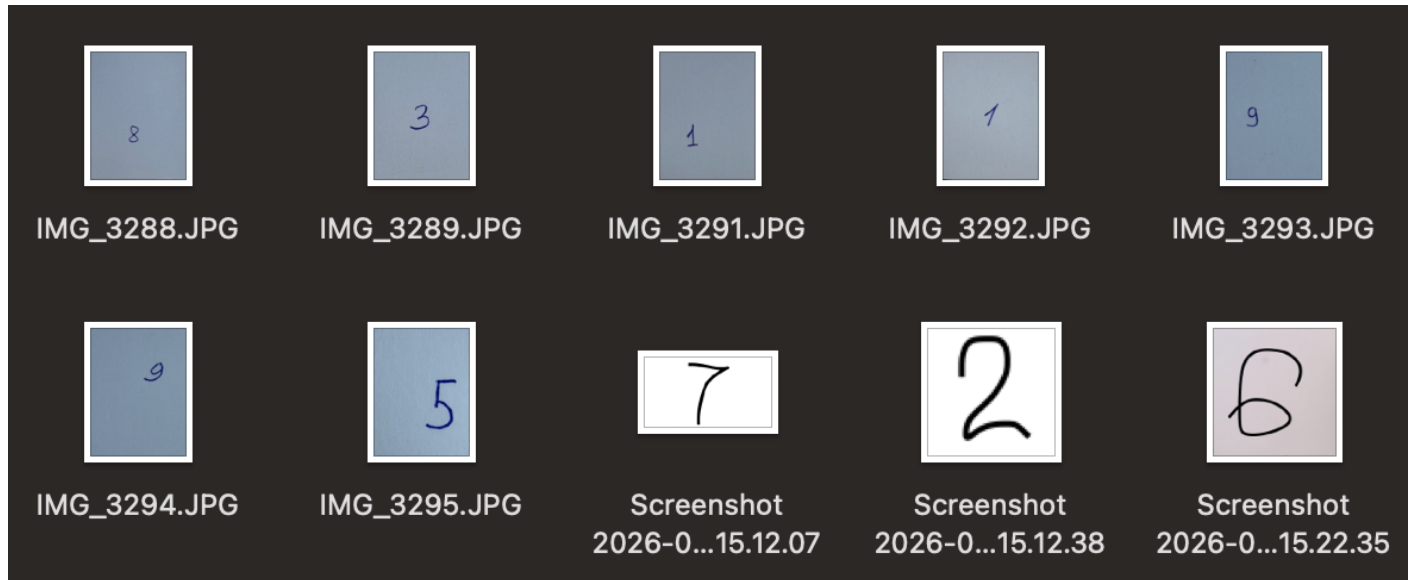


Рис. 5. – Початкові фото

Оскільки неймережа чутлива до масштабу та центрування, було застосовано конвеєр обробки зображень на базі бібліотеки **OpenCV**:

1. Зображення переводилося в градації сірого та бінаризувалося (чорний фон, біла цифра) за допомогою порогу (thresholding).
2. За допомогою пошуку контурів алгоритм автоматично знаходив межі намальованої цифри на великому фото та обрізав зайвий фон.
3. Для збереження пропорцій до обрізаної цифри додавалися чорні поля, утворюючи ідеальний квадрат.
4. Зображення масштабувалося до розміру 28x28 пікселів, нормалізувалося та подавалося на вхід неймережі.

Обробка фото...
Успішно оброблено картинок: 10

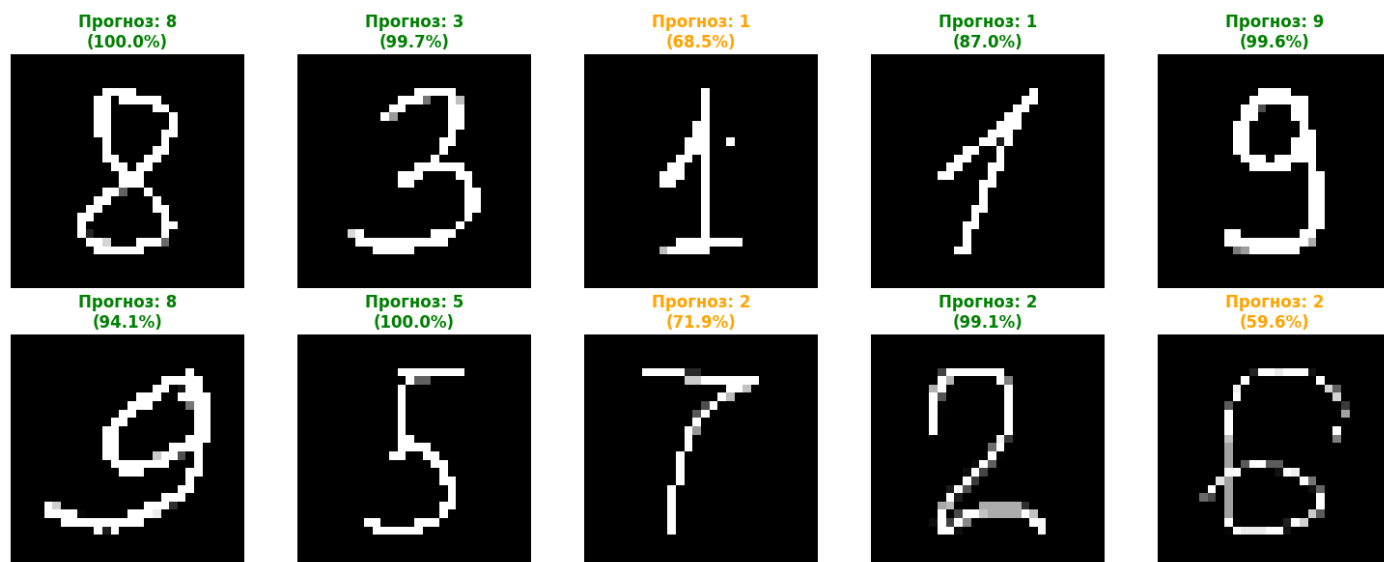


Рис. 5. – Розпізнані власні цифри

Модель непогано розпізнала завантажені фотографії власних цифр, що підтверджує коректність побудованої архітектури та правильність етапів підготовки даних.

Код:

```
import cv2
import os
import numpy as np
import matplotlib.pyplot as plt
import math

# 1. Функція що обробляє фото і передбачує
def get_digit_prediction(image_path, model):
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if img is None: return None

    _, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY_INV)
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not contours: return None

    c = max(contours, key=cv2.contourArea)
    x, y, w, h = cv2.boundingRect(c)
    digit_cropped = thresh[y:y+h, x:x+w]

    size = max(w, h)
    pad_w = (size - w) // 2
    pad_h = (size - h) // 2
    border = int(size * 0.2)
    padded_digit = cv2.copyMakeBorder(digit_cropped,
                                      pad_h + border, pad_h + border,
                                      pad_w + border, pad_w + border,
                                      cv2.BORDER_CONSTANT, value=0)

    resized_digit = cv2.resize(padded_digit, (28, 28))
    final_input = (resized_digit.astype('float32') / 255.0).reshape(1, 784)

    prediction = model.predict(final_input, verbose=0)
    predicted_class = np.argmax(prediction)
    confidence = np.max(prediction) * 100

    return resized_digit, predicted_class, confidence

folder_path = '/kaggle/input/datasets/darrriax/mynumbers/numbers/'
results = []

print("Обробка фото...")
for filename in sorted(os.listdir(folder_path)):
    if filename.lower().endswith(('.jpg', '.jpeg', '.png')):
        full_path = os.path.join(folder_path, filename)

        res = get_digit_prediction(full_path, model)
        if res:
            results.append(res)

print(f"Успішно оброблено картинок: {len(results)}\n")

n = len(results)
cols = 5
rows = math.ceil(n / cols)

plt.figure(figsize=(15, 3 * rows))

for i in range(n):
    img, pred, conf = results[i]

    plt.subplot(rows, cols, i + 1)
    plt.imshow(img, cmap='gray')

    color = 'green' if conf > 80 else 'orange'
    plt.title(f"Прогноз: {pred}\n({conf:.1f}%)", color=color, fontweight='bold')
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Відповіді на питання

1. Нейронна мережа прямого розповсюдження – це мережа, в якій нейрони не мають зв'язків між собою, якщо вони розташовані в одному шарі, але мають багато зв'язків з іншими нейронами з інших шарів. Її особливість в тому що вихідний сигнал попереднього шару стає вхідним наступного і так проходить поступово через всі шари, не повертаючись назад.
2. Багат шаровий персептрон має взідний шар з кількістю нейронів, що залежить від кількості вхідних даних. Далі оскільки він багат шаровий – то тут мінімум один прихований шар з функцією активації. І потім йде вихідний шар, який також залежить від кількості вихідних даних для задачі.
3. Інформація йде в одному напрямку за самою концепцією цієї мережі. Це робиться для того, щоб мережа працювала як чиста математична функція: отримала конкретні дані на вхід – видала конкретний результат на вихід. Якби були зворотні зв'язки, мережа зациклилася б і мала б стан «пам'яті» (як у рекурентних мережах).
4. Активаційна функція визначає вихідне значення нейрона на основі його зваженої суми. Її головна мета – внесення нелінійності в модель. Її не використовують у вхідному шарі, тому що вхідний шар – це якраз наші вхідні значення, які не мають змінюватися, а мають використовуватися в активаційній функції.
5. Зміщення (bias) – це рандомне числове значення на початку, що дає свободу нейрону змінювати це значення впродовж навчання. Воно зсуває графік активаційної функції вліво або вправо, дозволяючи нейрону активуватися раніше або пізніше.
6. Алгоритм зворотного поширення помилки – це метод обчислення градієнтів функції втрат для всіх ваг мережі. Він використовує математичне правило ланцюга, поширюючи інформацію про помилку від вихідного шару назад до вхідного, дозволяючи точно визначити, наскільки кожна окрема вага вплинула на загальну помилку. Для мереж прямого поширення це дає можливість корегувати ваги і досягати кращих значень передбачень.
7. Розпізнавання тексту, номерів авто, показників лічильника, класифікації табличних даних.
8. Нелінійність – властивість, що використовується в навчанні нейронних мереж, щоб дати мережі здатність розуміти складні, криволінійні залежності. Якби всі шари були лінійними, то хоч би скільки шарів ми не додали, вся мережа математично скоротилася б до рівняння однієї прямої лінії.

9. Тому що задача XOR – не лінійно сепарабельна задача. Ми не можемо провести площину в результуючих значеннях цієї задачі (потрібно шукати більш складні залежності), відповідно не можемо вирішити її за допомогою одношарового персептрона.
10. Кількість параметрів залежить від кількості шарів і нейронів в них. Якщо додаємо ще один шар – то кількість зв'язків між нейронами в шарах зростає. Складність зростає мультиплікативно. Якщо в одному шарі 100 нейронів, а в наступному 50, то між ними буде $100 \cdot 50 = 5000$ вагових коефіцієнтів, плюс 50 параметрів зміщення.
11. Простота реалізації, універсальність для будь-якої математичної задачі, швидкість прогнозування.
12. Не мають пам'яті (тому не запам'ятовують послідовності), фіксовані (якщо зсунути об'єкт на фото – то сприйматисеться як нова картинка).
13. Рекурентна має пам'ять.
14. Під час зворотного розповсюдження помилки значення градієнтів проходять через активаційні функції (особливо Sigmoid або Tanh) і постійно множаться на малі числа. У дуже глибоких мережах, поки помилка дійде до перших шарів, градієнт стає мікроскопічним (близьким до нуля). Через це перші шари просто перестають навчатися.
15. З перенавчанням, оскільки якщо мережа має занадто багато параметрів для простої задачі, вона замість пошуку закономірностей просто «зазубрює» тренувальні дані напам'ять. На нових даних така мережа покаже дуже поганий результат.

Висновки

Під час виконання лабораторної роботи було успішно розроблено, навчено та протестовано нейронну мережу прямого поширення для задачі розпізнавання рукописних цифр на базі класичного датасету MNIST. Було пройдено весь цикл машинного навчання: від попередньої обробки «сирих» даних до побудови архітектури моделі. Крім того, було реалізовано додатковий алгоритм на базі комп'ютерного зору для підготовки та розпізнавання власних фотографій написаних від руки цифр.

Результати тестування моделі виявилися високими – загальна точність склала 98%. Детальний аналіз метрик та матриці помилок показав, що мережа чудово засвоїла візуальні патерни і впевнено класифікує більшість символів. Незначна кількість хибних передбачень виникала переважно у випадках візуальної схожості символів (наприклад, плутанина між 7 і 2, або 5 і 6). У підсумку, створена модель довела свою ефективність не лише на стандартній тестовій вибірці, а й на реальних користувацьких зображеннях.